pyzdb：ZFS 盘上结构解析与可观测工具 - 技术说明书

项目源码：https://github.com/zhengyp36/pyzdb

第 1 章 背景与动机

ZFS 是一个高度复杂的文件系统，其设计中包含了完整而严密的盘上数据结构体系，包括 label、vdev、metaslab、objset、dnode、blkptr、zap、dsl_dir 等。
这些结构在代码中以大量 C 结构体和访问路径存在，在文档（如 zfs-on-disk）中以逻辑形式描述。
在实际工程中，我可以知道这些结构“应该是什么”，也可以通过阅读代码知道“系统是如何使用它们的”，但仍然存在一个核心问题：
我无法从真实磁盘中直接看到这些结构长什么样，也无法用盘上数据验证我对 ZFS 的理解是否正确。
当问题涉及空间分配异常、元数据损坏、跨层路径错误等场景时，仅依赖代码阅读和内核工具，很难建立一个完整的、可验证的模型。
这正是 pyzdb 的出发点。
pyzdb 的目标不是实现一个 ZFS，而是实现一个**从真实磁盘出发、把 ZFS 盘上结构还原为可观测对象模型的工具。**
它以磁盘为唯一数据源，从 ZFS label 开始，逐层解析 vdev、metaslab、objset、dnode 和 blkptr，最终可以从裸盘中读取出文件数据或内部结构。

第 2 章 设计原则

2.1 盘上数据是唯一真相源

pyzdb 不通过内核接口（如 ioctl、libzfs）获取任何 ZFS 结构信息。
所有数据都直接来自磁盘设备的物理内容。
每一个 ZFS 对象——无论是 dnode、zap 还是文件内容——都可以追溯到：
某个 vdev 上的某个偏移地址的一段字节序列。
这样做的目的，是使模型可以用于验证、排错和取证，而不仅是逻辑推演。

2.2 模型优先，而不是工具优先

pyzdb 并不是围绕“做一个命令”来设计，而是围绕“构建一个可扩展的盘上模型”来设计。
ZFS 盘上结构的本质是一组字段定义（offset + type + size），只要这些定义被正确描述，就可以从裸数据中解析出对象。

因此，pyzdb 引入了通用结构解析模型（CStruct），使新对象的引入不依赖大量解析代码，而只依赖字段描述。

2.3 Python + C 的混合架构

pyzdb 采用 Python 与 C 的混合架构：
Python 用于

-
  - 对象建模
  - 结构组织
  - 交互式探索

C 用于

-
  - 磁盘 mmap 访问
  - checksum、解密、解压
  - xdr / raidz 运算

这样可以在保持建模灵活性的同时，复用 ZFS 中成熟的底层算法实现。

第 3 章 总体架构

pyzdb 从物理磁盘到 ZFS 对象模型分为四层：

```
ZFS Object Layer   (dnode, objset, zap, metaslab)
Block Layer        (blkptr, dva, checksum, decrypt)
Disk Layer         (vdev, raidz, mmap IO)
Physical Disk
```

数据流从下往上：

1. Disk Layer 从磁盘读取原始字节
2. Block Layer 根据 blkptr 描述重建并验证数据块
3. Object Layer 将数据块解释为 ZFS 结构对象

从 label 开始，核心路径为：
label → nvlist → spa → vdev → metaslab → objset → dnode → blkptr → data
这一条路径覆盖了从 pool 到文件内容的完整链路。

第 4 章 磁盘与 Label 解析

pyzdb 首先扫描系统中的块设备（如 /dev 下的磁盘），并对每个设备执行 mmap，将整个设备映射为内存。
ZFS 的每个磁盘包含四个 label 区域，内部以 xdr 编码的 nvlist 形式保存 pool 与 vdev 的元数据。
pyzdb 通过移植并实现 xdr / nvlist 解析逻辑，从每个磁盘的 label 中读取出：

- pool GUID
- vdev GUID

- 拓扑关系
- 设备角色（mirror、raidz、spare 等）

通过对所有磁盘 label 的交叉匹配，pyzdb 构建出完整的 pool 与 vdev 树结构，对应 ZFS 中的 SPA。

第 5 章 blkptr 与数据访问模型

blkptr（block pointer）是 ZFS 盘上结构的核心，它同时描述：

- 数据块所在位置（DVA）
- 校验算法与校验值
- 压缩方式
- 加密方式
- 逻辑大小与物理大小

在 pyzdb 中，blkptr 被封装为一个可调用对象，其主要职责是：

1. 根据 DVA 定位到 vdev 与偏移
2. 从磁盘读取原始块
3. 对 raidz/mirror 进行数据重建
4. 校验 checksum
5. 执行解密
6. 执行解压

最终返回一段可用于上层对象解析的字节数据。

通过这一封装，所有 ZFS 对象都可以统一通过 blkptr 访问，而无需关心底层物理布局。

第 6 章 CStruct 与对象建模

ZFS 盘上结构的一个特点是：

它们不是树形编码，而是定长或半定长的字段布局。

pyzdb 引入 CStruct 作为通用结构解析基类。

每一个 ZFS 对象只需要描述：

- 字段名
- 偏移
- 类型（u64, array, embedded struct 等）

CStruct 会自动从原始字节中解析出这些字段，并暴露为 Python 属性。

例如，一个 dnode 或 blkptr 并不需要单独编写解析代码，只需要定义其字段布局。

这样，当遇到新的 ZFS 结构时，只需要添加字段定义即可扩展模型。

第 7 章 核心 ZFS 对象

在 pyzdb 中，已实现的主要对象包括：

- objset：表示一个 ZFS 对象集，如 dataset 或 meta-objset
- dnode：表示一个对象的元数据与数据指针
- zap：表示 ZFS 内部的键值存储
- metaslab：表示磁盘空间分配状态

- spacemap：表示空间变化日志

这些对象都由 blkptr 引导，从磁盘中逐级解析而来。
例如，从 root_bp 解析出 meta-objset，再从 meta-objset 中解析出各个 objset，再由 objset 中的 dnode 解析文件、目录和内部结构。

第 8 章 从 Pool 到文件

通过上述模型，可以完成从 pool 到文件内容的完整路径：

1. 通过 label 打开 pool
2. 定位 root dataset
3. 通过 zap 查找文件对应的 object number
4. 通过 dnode 获取文件的 blkptr
5. 通过 blkptr 读取文件数据

这一过程不依赖 ZFS 内核，而完全基于盘上结构。

第 9 章 工程价值

pyzdb 的主要价值在于：

- 验证 zfs-on-disk 文档的实际含义
- 验证源码中对盘上结构的假设
- 分析空间分配与回收行为
- 在损坏场景下检查元数据
- 为 ZFS 内部结构提供可观测模型

它更接近“盘上结构调试器”，而不是管理工具。

第 10 章 局限与下一步

pyzdb 并未覆盖 ZFS 的全部特性，例如部分新版本特性与高级压缩/加密模式尚未完整支持。
性能也并非主要目标，其设计更偏向于可观测性与正确性。
未来可以扩展的方向包括：

- 更完整的 feature flag 支持
- 与 zdb 输出的对齐验证
- 对损坏场景的更好诊断支持